implementation, *deployment*, and *temporal* coupling.[1] Let's consider the Polar Bookshop system we've been working on so far.

We can change the implementation of any of the applications without having to change the others. For example, we can re-implement Catalog Service using the reactive paradigm without affecting Order Service. Using a service interface like a REST API, we hide the implementation details, improving loose coupling. All the applications can be deployed independently. They're not coupled, reducing risks and increasing agility.

However, if you think about how the applications we built so far interact, you'll notice that they need other components of the system to be available. Order Service needs Catalog Service to ensure that a user can order a book successfully. We know that failures happen all the time, so we adopted several strategies to ensure resilience even in the face of adversity, or at least ensuring a graceful degradation of functionality. That's a consequence of *temporal coupling*: Order Service and Catalog Service need to be available at the same time to fulfill the system requirements.

Event-driven architectures describe distributed systems that interact by *producing* and *consuming* events. The interaction is asynchronous, solving the problem of temporal coupling. This chapter will cover the basics of event-driven architectures and event brokers. You'll then learn how to implement business logic using the functional programming paradigm and Spring Cloud Function. Finally, you'll use Spring Cloud Stream to expose the functions as message channels via RabbitMQ, building event-driven applications through the publisher/subscriber (pub/sub) model.

NOTE The source code for the examples in this chapter is available in the Chapter10/10-begin, Chapter10/10-intermediate, and Chapter10/10-end folders, containing the initial, intermediate, and final state of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).